

ELE344 Conception et architecture de processeurs

Tutoriel : Le Simulateur MARS

MARS est un simulateur logiciel du langage assembleur destiné à l'architecture MIPS. Il a été développé par Pete Sanderson et Kenneth Vollmar à l'université «*Missouri State University*» Nous explorerons dans ce tutoriel les fonctionnalités du simulateur. Vous trouverez fichier d'installation du simulateur MARS sur Moodle dans la section Projets. Vous pouvez aussi le télécharger à partir de site web www.cs.missouristate.edu/MARS.

À travers l'exemple 1, nous vous montrons les fonctionnalités de bases de MARS. Vous trouverez plus de fonctionnalités dans la dernière section **Astuces**.

Exemple 1 : addi \$2,\$0,5

1. Télécharger le simulateur MARS à partir du Moodle ou du site cité ci-dessus.
2. Lancer le simulateur, vous devez avoir l'interface du simulateur présenté à la figure 1

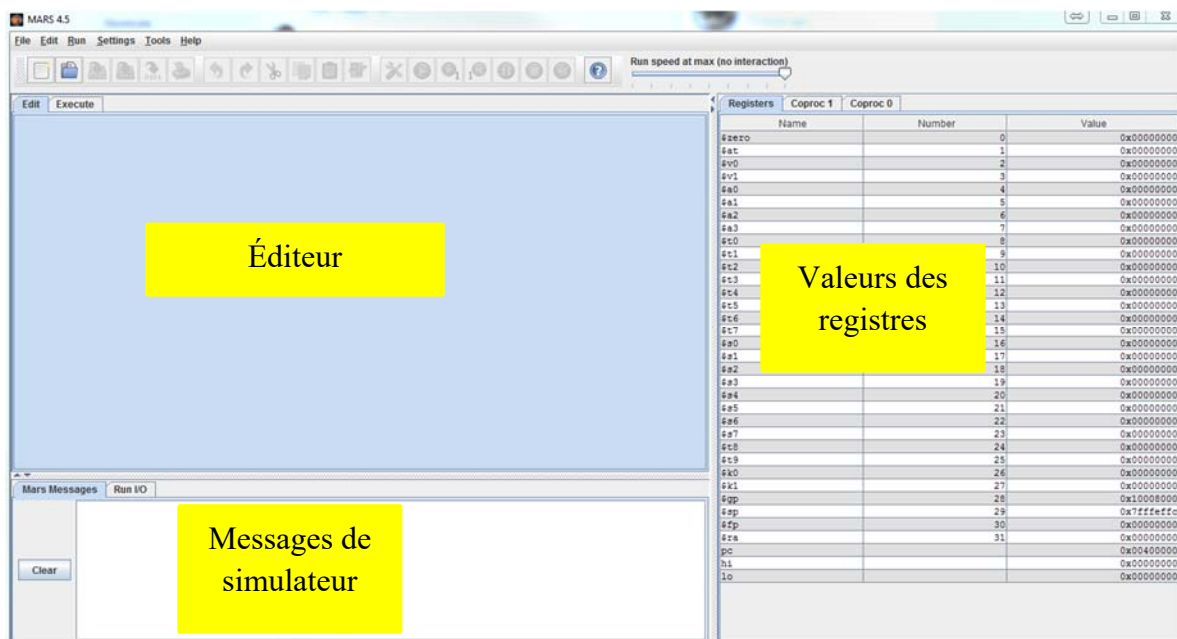


Figure 1: Interface du simulateur MARS

La barre d'outils du MARS est présentée à la Figure 2.



Figure 2: barre de menus

3. Créer un nouveau fichier : Aller sur **FILE** / **new**
4. Écrivez dans le nouveau fichier : `addi $2,$0,5`
5. Enregistrer le fichier sous `addi.asm`
6. Configurer la mémoire : **Settings** / **Memory configuration** comme présenté à la Figure 3. Assurez-vous que **Text at Address 0** est coché puis Appuyez sur **Apply** and close.

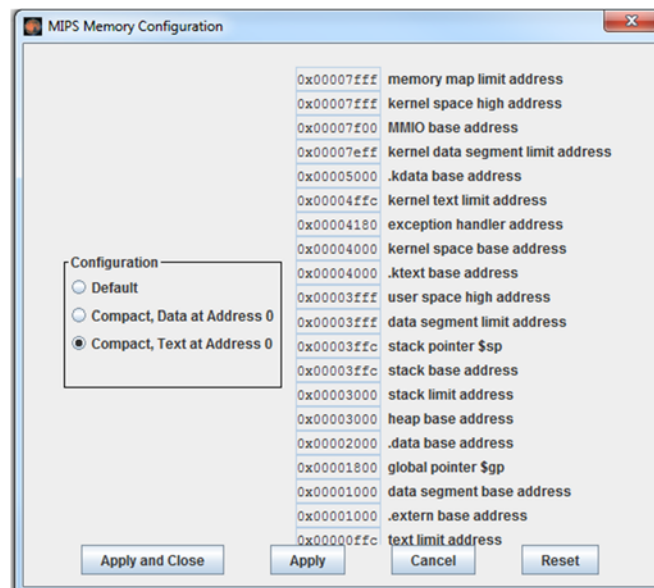


Figure 3 : Configuration de la mémoire

7. Allez sur **Run** / **Assemble** ou appuyer directement sur le bouton  en haut à droite dans la barre des menus.
8. Une fois la compilation est complétée avec succès, la fenêtre «*Execute*» apparaîtra (Figure 4).

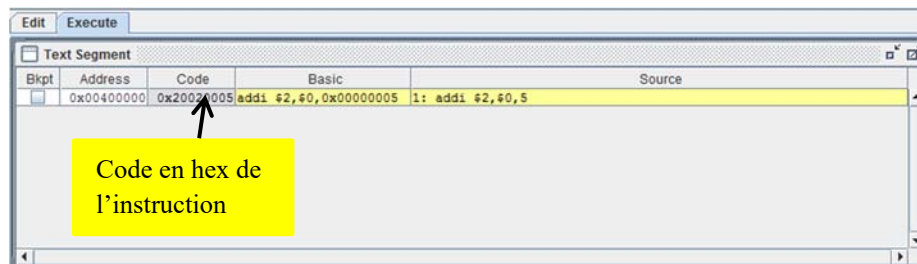


Figure 4: Interface Execute

Si la compilation est réussie, vous obtiendrez dans la zone des messages : *Assemble: operation completed successfully*.

N.B. Si vous avez fait une erreur dans votre code, le panneau de l'éditeur affichant votre code restera visible et la première erreur sera soulignée dans la fenêtre. Un message d'erreur décrivant le problème apparaîtra dans la zone des messages suivi de l'avertissement : «*Assemble: operation completed with errors*». Dans ce cas, vous devez examiner et corriger votre code, puis réessayer.

Une fois que l'assembleur a réussi, une flèche verte (sigle Play) située immédiatement à droite du bouton d'assemblage devient active. Cliquez sur ce bouton pour exécuter votre programme. Un message apparaîtra dans la fenêtre des messages «*Mars message*» qui dit : «*program is finished running*». Après, vous pouvez remarquer que le registre 2 change de valeur de 0 à 5.

9. Exportez le code binaire : Vous pouvez générer le code hexadécimal (ou binaire) des instructions en cliquant sur **FILE / Dump Memory / Hexadecimal Text** (Fig 5).

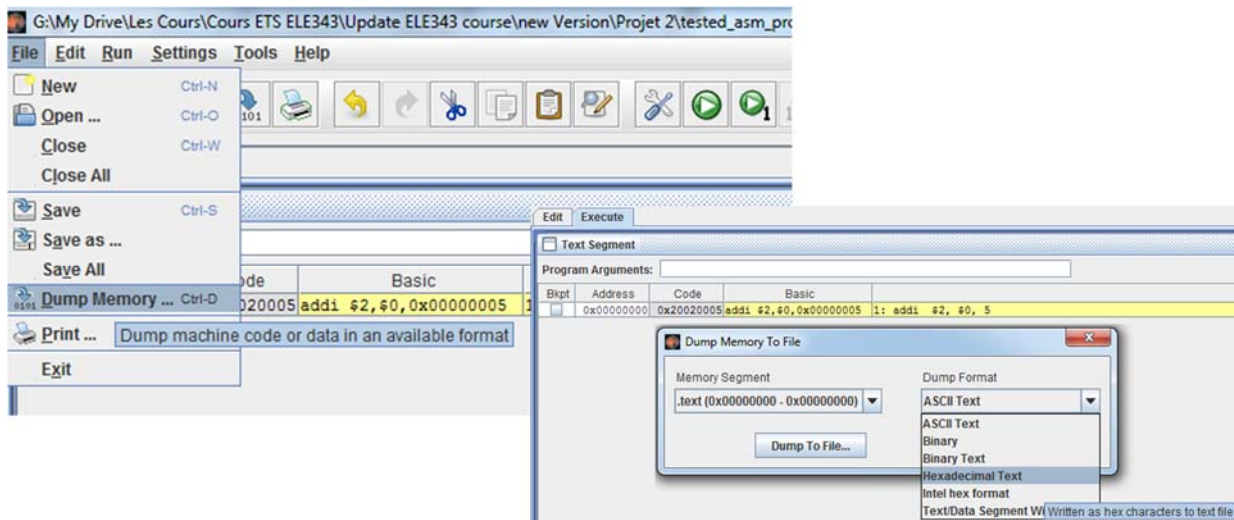


Figure 5 : Génération du code binaire des instructions

Exécuter les exemples suivants pour mieux familiariser avec MARS.

Exemple 2 : Traitement des opérations : IF et ELSE

```
# Programme if C then T else F
# if ($1 < $2) then // C
#   $3 = $2 + $1; // T
# else $3 = $2 - $1; // F
# Initialisation des registres
addi  $1, $0, 2
addi  $2, $0, 1
# Programme if C then T else F
slt   $4, $1, $2      # C
beq   $4, $0, ELSE
```

```

    add    $3, $2, $1    # T
    j      ENDIF         # T
ELSE:    sub $3, $2, $1    # F
ENDIF:

```

Exemple 3: Boucle for

```

# Boucle itérative FOR (I;C;A) { S }
#
# for (i=0;i<100;i++)
#   {a = a + i;}
    addi    $1,$0,0        # I i=$1
BOUCLE:    slti    $3,$1,100 # C
    beq     $3,$0, FIN     # C
    add     $2,$2,$1       # S a=$2
    addi    $1,$1,1        # A i++
    j      BOUCLE
FIN:

```

Exemple 4: Load word / Store word (Lw et Sw)

```

    addi $7, $0, 12
    addi $3, $0, 12
    j    next
    addi $2, $0, 7
next:    sw  $7, 8180($3)
        lw  $2, 8192($0)
        sw  $2, 8196($0)

```

Exemple 5: Calcul du PGCD

```

# Programme PGDC (Plus grand dénominateur commun)
# while ($1 != $2) {
#   if ($1 > $2 )
#     $1 -= $2;
#   else
#     $2 -= $1;
# }
# Initialisation des registres
    addi    $1, $0, 12
    addi    $2, $0, 8
# Programme
BOUCLE:    beq    $1,$2,FIN
    addi    $4,$4,1
    slt     $3,$2,$1
    beq     $3,$0,ELSE
    sub     $1,$1,$2
    j      BOUCLE
ELSE:      sub     $2,$2,$1
    j      BOUCLE
FIN:

```

N.B Vous trouverez des autres exemples dans le site de cours sur Moodle sous l'onglet Exercice / Exemples de code MIPS.

Vérification :

Simuler le code présenté dans le document code_mips.docx. Exécuter et générer le code hexadécimal des instructions. Lorsque votre vérification du bon fonctionnement du code est terminée, préparer toutes les fenêtres (edit, execute et le code héxa) et faire vérifier les résultats par votre chargé de laboratoire.

Astuces

1. Ralentissement de l'exécution

Vous pouvez utiliser la barre de défilement (Figure 6) pour modifier la vitesse d'exécution à environ 1 instructions par seconde. Cela nous permet de regarder l'exécution du programme pas à pas au lieu qu'il se termine directement d'un seul coup.

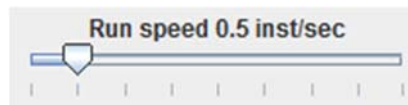


Figure 6: Vitesse exécution

2. Exécution pas à pas

Pour mieux comprendre l'exécution votre programme assembleur ainsi que les registres modifiés, vous pouvez exécuter pas à pas le programme, c'est-à-dire instruction par instruction. Pour ce faire, il suffit d'appuyer sur le bouton d'exécution pas à pas au lieu d'exécuter tout le programme (Figure 7). Ce mode d'exécution vous permet de déboguer votre programme.

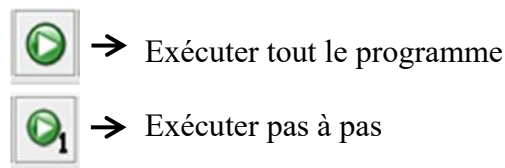


Figure 7 : Exécution pas à pas

3. Outils : MIPS X-Ray

Cet outil permet de visualiser l'exécution d'une instruction ainsi que la propagation des bits dans l'architecture du processeur. Pour lancer X-ray cliquer respectivement sur :

- *Tools / MIPS X-Ray*
- *Connect to MIPS*
- Pour passer d'une instruction à une autre, appuyer sur le bouton d'exécution pas à pas en haut de la fenêtre.

La Figure 8 vous montre l'exécution de l'exemple 1, présenté au début dans ce tutoriel.

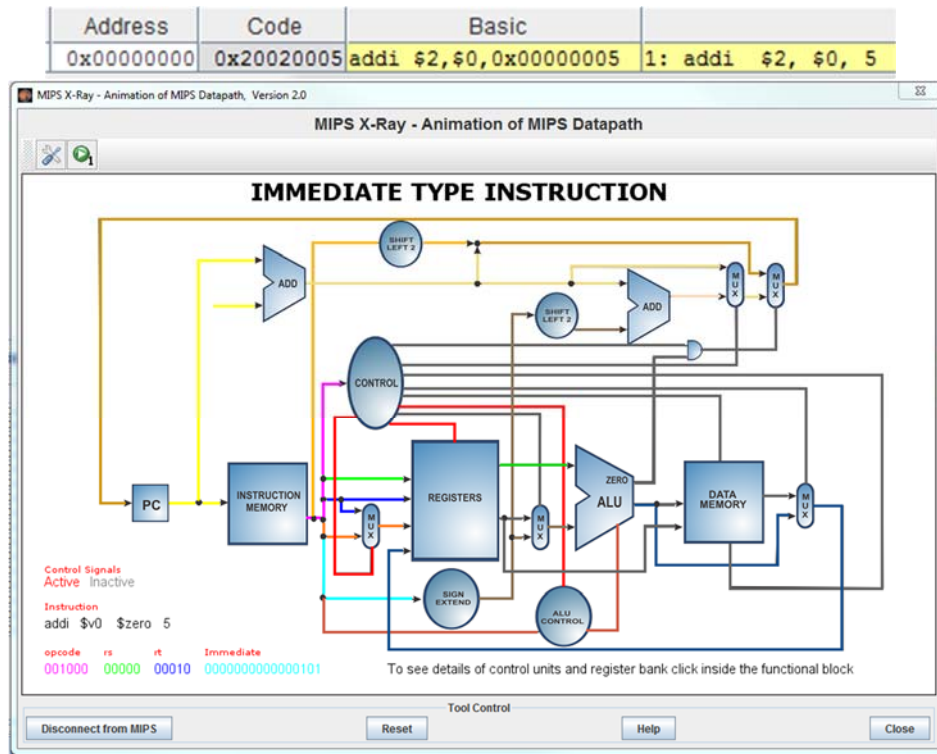


Figure 8 : Outil MIPS X-Ray